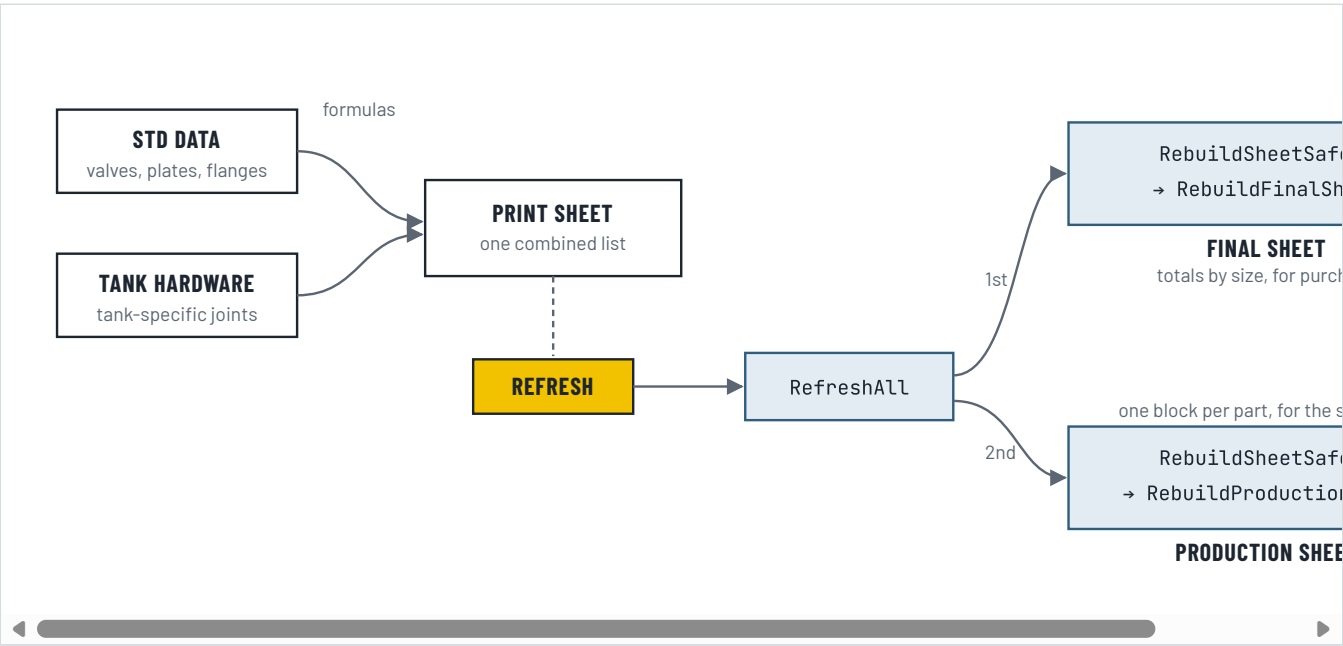


HARDWARE LIST MACROS How the REFRESH button builds FINAL SHEET and PRODUCTION SHEET, how each macro works line by line, and how to change one yourself without breaking the workbook.	WORKBOOK	6367_HARDWARE LIST_PRODUCTION.xlsm
	FORMAT	F/DSN/54 · V1.4
	MODULES	3 (+ VERIFICATION)
	WRITTEN	09.10.26

01 THE ONE-BUTTON FLOW

The designer fills in **TANK HARDWARE** and **STD DATA**. STD DATA also holds the job details and the hardware material, such as HDG or SS, for sizes up to M12 and above M12. PRINT SHEET follows by itself, because it is all formulas. Then they press **REFRESH** on PRINT SHEET. REFRESH first re-runs PRINT SHEET's "hide zero rows" filter, so a joint that was just switched on appears there too. That one press rebuilds both summary sheets from the same list, so their totals always agree. When it finishes, one message says either "FINAL SHEET and PRODUCTION SHEET are up to date", or what went wrong on which sheet.



TWO WAYS IN. The REFRESH button is one. The print shortcuts are the other:
Ctrl + Shift + P (print) and **Ctrl + Shift + Q** (print protected). Either shortcut rebuilds FINAL SHEET or PRODUCTION SHEET first if that's the sheet you are printing. If the rebuild hits a problem, the macro shows the problem and doesn't print, so stale paper never reaches the shopfloor.

02 **WHERE THE NUMBERS COME FROM**

Since the October 2026 version, the input is split over two sheets. PRINT SHEET is the only sheet that has every joint from both. That's why both macros read PRINT SHEET and never TANK HARDWARE directly. A macro that read TANK HARDWARE alone would miss every valve, plate and flange joint.

PRINT SHEET COLUMN	HOLDS	USED BY PRODUCTION SHEET AS
D	Joint location (merged down when one joint uses several parts)	JOINT LOCATION
F	Part name, e.g. M12x50-HDG or M12-STUDx75-HDG	The block it goes in
G	Bolt / stud quantity	BOLT or STUD
H	Nut	NUT
I	Plain washer	P. WASHER
J	Spring washer	S. WASHER
K	Lock nut	L.NUT
L	Standard reference, e.g. SD-269	STD.REF

This is what one block looks like after a refresh, using real rows from WO 6367:

M10X50-HDG						
JOINT LOCATION	BOLT	NUT	P. WASHER	S. WASHER	L.NUT	STD.REF
OLTC CONSERVATOR	18	18	36	0	0	SD-268
FAN BRACKET MOUNTING TO TANK	22	22	44	22	0	SD-269

TOTAL	40	40	80	22	0	
-------	----	----	----	----	---	--

03 EVERY MACRO AND ITS JOB

A **module** is a page of code inside the workbook. A **macro** (a Sub) is one named job on that page. These are the ones you will meet:

MODULE	MACRO	WHAT IT DOES	WHO RUNS IT
ProductionSheetBuilder	RefreshAll	Rebuilds FINAL SHEET, then PRODUCTION SHEET, then shows one message: all up to date, or what went wrong.	REFRESH button on PRINT SHEET
ProductionSheetBuilder	RebuildSheetSafely	Unlocks a protected sheet, runs its rebuild, locks it again. Answers True or False: did the rebuild work?	RefreshAll and the print macros
ProductionSheetBuilder	RebuildProductionSheet	Wipes PRODUCTION SHEET from row 7 down and writes one block per part.	RebuildSheetSafely
FinalSheetBuilder	RebuildFinalSheet	Writes the purchase totals: nuts and washers by size, then bolts, then studs.	RebuildSheetSafely
Module1	PRINTOUTPUT	Rebuilds the sheet if needed, then prints it. Ctrl + Shift + P	You

Module1	PRINTPROTECTEDOUTPUT	Same, but locks the sheet before printing. <code>Ctrl + Shift + Q</code>	You / PRINT OUTPUT PAGE button
VerificationBuilder	RunVerification	Checks every joint on TANK HARDWARE <i>and</i> STD DATA against the joint standard: nut and washer counts per bolt, lengths, and whether the totals agree from the input sheets through PRINT SHEET to FINAL SHEET. Findings name the sheet and row, e.g. TH 57 or SD 50 . Unused joints (all zero) are skipped.	VERIFY button on VERIFICATION

WHY "SAFELY"? A protected print (`Ctrl + Shift + Q`) leaves the sheet locked, and Excel won't let a macro write to a locked sheet. Without this helper, the next REFRESH would stop with error 1004. RebuildSheetSafely unlocks the sheet, rebuilds it and locks it again, so you never notice. It only removes protection that has no password. A sheet locked with a password is left alone, with a message, and Excel never asks for the password.

04 PRODUCTION SHEET STEP BY STEP

Open `ProductionSheetBuilder` in the VBA editor and follow along. The steps below match the numbered comments in the code. Steps 1 to 4 only *read*. Nothing is changed until step 5, so if a check fails, the sheet stays exactly as it was.

0

SAVE EXCEL'S SETTINGS AND TURN ON THE SAFETY NET

The macro switches off screen redraw to run fast. It remembers the old settings first, so it can always put them back. `On Error GoTo Fail` means "if anything unexpected goes wrong, jump to the `Fail:` label at the bottom". That label restores Excel and shows the error.

```
' Remember Excel's settings so we can put them back at the end
prevScreen = Application.ScreenUpdating
prevEvents = Application.EnableEvents
prevCalc = Application.Calculation

On Error GoTo Fail          ' any unexpected error jumps to Fail:
```

1

FIND HOW FAR PRINT SHEET GOES

The macro never assumes "the list ends at row 245". It takes the bottom of the sheet's used area as the end of the list. Rows the designers add later are picked up automatically.

```
LastRow = LastSourceRow(ps)

' inside LastSourceRow: the bottom of the used area, filtered rows included
LastSourceRow = ps.UsedRange.Row + ps.UsedRange.Rows.Count - 1
```

WHY NOT CTRL+UP? PRINT SHEET has a filter that hides joints with zero quantity. The quicker `End(xlUp)` (the macro version of `Ctrl + Up`) jumps over hidden rows. So when a designer switched on the foundation bolts at the very bottom of the list, both sheets silently left them out. `Find` skips filtered rows too. `UsedRange` doesn't care about filters, and rows below the list (notes, signatures) have no part name, so they are skipped anyway. A good rule for this workbook: never use `End(xlUp)` or `Find` to find the end of a filtered sheet.

2

READ EVERY ROW AND FILE IT UNDER ITS PART

For each row, the macro reads the five quantities. If they are all zero, nothing is

fitted there on this job, so the row is skipped. Otherwise the row goes into a **Dictionary**, which works like a filing cabinet: the drawer label is the part name, and the drawer holds the list of rows for that part.

```
For c = 1 To 5
    qty(c) = Num(ps.Cells(r, 6 + c).Value) ' G, H, I, J, K
    If qty(c) <> 0 Then anyQty = True
Next c

key = PartKey(re, CStr(desig)) ' "M12x50-HDG" → "M12X50-HDG"
If Not groups.Exists(key) Then groups.Add key, New Collection
groups(key).Add rec ' put this row in that drawer
```

The joint location comes from `JointLocation`. On PRINT SHEET, one joint name can cover several rows (the cell is merged downwards), and only the top cell holds the text. `MergeArea.Cells(1, 1)` always reads that top cell.

MATERIAL STAYS IN THE NAME. M12x60-SS and M12x60-HDG are different drawers. Stainless bolts must never be added to galvanised ones.

3 STOP IF ANY PART NAME CAN'T BE READ

Every name must look like M12x50-HDG or M12-STUDx75-HDG. The macro checks this with a **regular expression**, a pattern for text. If even one name fails, it lists up to 8 bad rows and stops *before* changing anything.

```
re.Pattern = "^\\s*(M\\d+)\\s*(-STUD)?\\s*x\\s*(\\d+)\\s*-\\s*(.+?)\\s*$"
'           size   stud marker   length   material
```

4 PUT THE PARTS IN SHOPFLOOR ORDER

Part names are first tidied into one form (m12 x050-hdg becomes M12X50-HDG), the same way FINAL SHEET reads them. The order is all bolts first, then studs. Within each, smallest size first, then shortest length. Plain text sorting would put M10 before M8, because "1" comes before "8". So each name gets a sort code with zero-padded numbers, and the codes are sorted instead.

```
' bolt+ M8x40-HDG → "0100081000401HDG"
```


Excel has no "keep together" setting, so the macro asks Excel where each page would end. If a page would end inside a block, it puts a page break just above that block's yellow title, so the whole block moves to the next page. It then checks again, because moving one block shifts every page after it.

```
ws.PageSetup.PrintArea = "$D$2:$K$" & lastWritten
ws.PageSetup.PrintTitleRows = PD_TITLE_ROWS      ' "$2:$6"

' inside KeepBlocksTogether, for each page end Excel plans:
pageRow = ws.HPageBreaks(i).Location.Row
If starts(b) < pageRow And blockEnd ≥ pageRow - SAFETY_ROWS Then
    ws.HPageBreaks.Add Before:=ws.Cells(starts(b), "D")    ' new page at the
End If
```



05 VBA BASICS USED HERE

You only need a handful of ideas to read every line of these macros.

SUB AND FUNCTION

A Sub does a job(Sub RefreshAll()). A Function does a job and hands back an answer(Num(v) returns a number).

DIM AND OPTION EXPLICIT

Dim r As Long creates a variable. Option Explicit at the top makes VBA refuse undeclared names, which catches typos like lastRw .

COMMENTS

Anything after a single quote ' is a note for people. VBA ignores it. The editor shows comments in green.

CELLS AND RANGE

ws.Cells(10, "F") is cell F10.
ws.Range("D7:K7") is a block of cells.
.Value reads or writes what's in them.

IF AND FOR

If x < 0 Then ... decides. For r = 9 To lastRow ... Next r repeats for every row.

DICTIONARY AND COLLECTION

A Dictionary is a lookup by name ("M12X50-HDG" → rows). A Collection is a simple list you .Add to.

WITH ... END WITH

CONSTANTS

A shorthand: inside With rng , writing
.Font.Size = 16 means rng.Font.Size
= 16 .

Const PD_FIRST_ROW As Long = 7 names
a fixed value once. Change it in one place
instead of hunting through the code.

ERRORS

On Error GoTo Fail sends any crash to a
cleanup label. MsgBox shows a message to
the user.

06 WORKING IN THE VBA EDITOR

TO DO THIS	DO THIS
Open the editor	<code>Alt + F11</code> . The left panel lists modules under <i>Modules</i> . Double-click one to read it.
Show the Developer tab	File > Options > Customize Ribbon > tick Developer .
Run a macro	<code>Alt + F8</code> , pick it, Run. Or click inside a Sub in the editor and press <code>F5</code> .
Watch it run line by line	Click inside the Sub and press <code>F8</code> repeatedly. The yellow line is the next one to run. Switch to Excel to see the effect.
Pause at one line	Click the grey margin beside a line (or press <code>F9</code>) to set a red breakpoint. <code>F5</code> runs up to it.
Check a value while paused	Hover over a variable, or open the Immediate window (<code>Ctrl + G</code>) and type ? lastRow then Enter.
Stop a running macro	Run > Reset (the blue square). <code>Esc</code> or <code>Ctrl + Break</code> interrupts a long loop.
Check for typos before running	Debug > Compile VBAProject. No message means no errors.
Attach a macro to a button	Developer > Insert > Button (Form Control), draw it, then pick the macro. To change it later: right-click the button > Assign Macro.

NO UNDO AFTER A MACRO. `Ctrl + Z` can't reverse what a macro did. Save first. If a rebuild goes wrong, close without saving.

07 MAKING COMMON CHANGES

YOU WANT TO	CHANGE THIS
Rename PRODUCTION SHEET or PRINT SHEET	The constants <code>PROD_SHEET</code> / <code>SRC_SHEET</code> at the top of <code>ProductionSheetBuilder</code> . Also update the two lines in <code>Module1</code> and the name in <code>RefreshAll</code> .
Start the blocks on a different row	<code>PD_FIRST_ROW</code> (now 7). Keep it below the header rows.
Repeat different header rows on each page	<code>PD_TITLE_ROWS</code> (now <code>"\$2:\$6"</code>).
Change the title-bar colour	<code>.Interior.Color = RGB(255, 255, 0)</code> in <code>WriteBlock</code> . Red, green and blue, each 0–255.
Show joints with all-zero quantities too	In step 2, replace <code>If anyQty Then</code> with <code>If True Then</code> . Expect a much longer printout.
Change the order of the blocks	The sort code in <code>SortedPartKeys</code> . The first part, <code>"0" / "1"</code> , is bolt or stud. Then size, length and material.
A TOTAL row still spills onto the next printed page	<code>SAFETY_ROWS</code> in <code>KeepBlocksTogether</code> (now 2). Raise it by 1. Each printer measures rows slightly differently, and this is the spare room left at the bottom of each page.
Change the page-number text	Page Layout › Print Titles › Header/Footer › Custom Footer. It is currently <code>&8Page &P of &N : 8pt</code> text, this page, total pages.
Add a new REFRESH button somewhere else	Developer › Insert › Button, then assign <code>RefreshAll</code> .

08 SAFETY RULES

1. **Keep a copy before editing code.** Save the workbook under a new name first.
2. **Always save as .xlsm.** Saving as .xlsx deletes every macro without asking twice.

3. **Enable macros on open.** If the yellow "Enable Content" bar shows, the buttons do nothing until you press it.
 4. **Never delete a Sub with code.** One attempt to delete a single macro by code once wiped a whole module. To remove a macro, select its text in the editor and delete it by hand.
 5. **Compile after every edit.** Run Debug > Compile VBAProject, then test on a copy before using it on a real work order.
 6. **Read the message.** Every guard says "Nothing has been changed" when it stops. If you see that, fix the row it names and press REFRESH again.
-

09 MESSAGES AND FIXES

MESSAGE	WHAT IT MEANS	FIX
could not read N part name(s) on PRINT SHEET	A part name in column F doesn't follow M12x50-HDG , or shows #REF! .	Go to the row it lists and correct the input on STD DATA or TANK HARDWARE.